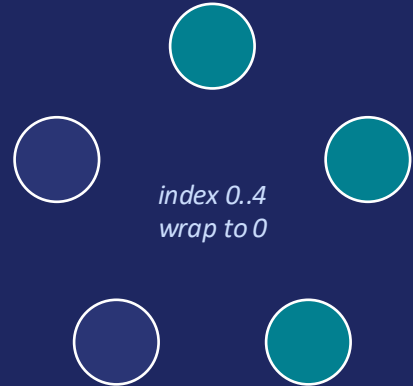


The Circular Queue

Fixing False Overflow — Wrap Around Instead of Running Off the Edge

Unit II • Lecture 13



Recap: the problem we left unsolved

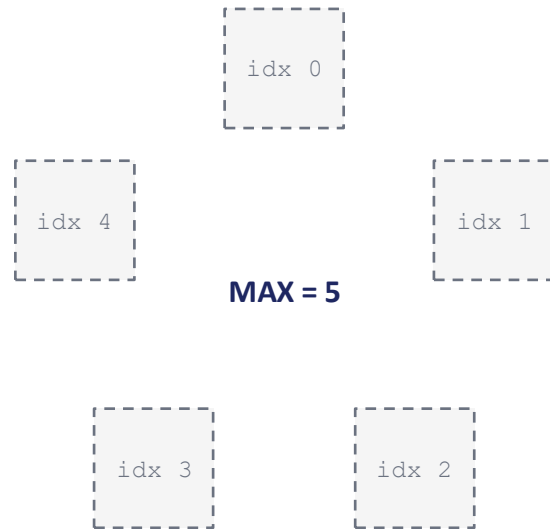
From Lecture 11

front = 2, rear = 4 in a queue of size 5. Two slots (0 and 1) are empty. But enqueue(60) still says "Queue Overflow" — because rear has hit the last index and never comes back.

- The array never had a real shortage of space — the space was just on the WRONG side.
- front and rear only ever moved right ($\text{rear} = \text{rear} + 1$), so once rear reaches the end, it is stuck.
- Today's fix is one idea: let rear (and front) come back to slot 0 after the last slot. That is a circular queue.

The idea: an array bent into a circle

- Physically the array is still a straight row: `queue[0]` to `queue[MAX-1]`.
- We only change how we THINK about the last slot: after `queue[MAX-1]` comes `queue[0]` again — like a clock face.
- front and rear now move in a loop, not a straight line.
- No new memory, no new struct — just a different rule for 'what comes next'.



The wrap rule: one operator does it all

```
index = (index + 1) % MAX
```

- % is the remainder (modulo) operator: $a \% b$ is what is left after dividing a by b .
- When index is less than $\text{MAX} - 1$, adding 1 and taking % MAX changes nothing — it behaves exactly like before.
- The moment index reaches $\text{MAX} - 1$, adding 1 gives MAX, and $\text{MAX} \% \text{MAX} = 0$ — it wraps to the start automatically.

The tricky part: how do we know it's FULL now?

Old rule (wrong here)

`rear == MAX - 1`

Fails immediately: rear now legally revisits every index, including `MAX - 1`, while NOT full.

New rule

`(rear + 1) % MAX == front`

"If the NEXT slot rear would move into is where front is standing, there is no room left."

- Full now means: rear has looped all the way around and is about to bump into front.
- This is the one genuinely new idea in circular queues — everyone gets it wrong on the first try.
- `isEmpty` does NOT change: `front == -1` still means empty, exactly as before.

The code: only two lines actually change

```
#include <stdio.h>
#define MAX 5

int queue[MAX];
int front = -1;
int rear  = -1;

int isEmpty(void)
{
    return front == -1;
}

int isFull(void)
{
    return (rear + 1) % MAX == front;
}
```

- Declarations: identical to Lecture 11.
- isEmpty: identical to Lecture 11.
- isFull: the ONE line that is genuinely new
- boxed on the previous slide.

Reassuring fact: most of Lecture 11 survives unchanged.

enqueue() — the other line that changes

```
void enqueue(int x)
{
    if (isFull())
    {
        printf("Queue Overflow\n");
        return;
    }

    if (front == -1)
        front = 0;

    rear = (rear + 1) % MAX;
    queue[rear] = x;
}
```

- Guard, first-element wake-up: identical to Lecture 11.
- `rear = rear + 1` becomes `rear = (rear + 1) % MAX` - the only change in the whole function.
- That one % is what lets rear reuse a freed slot instead of refusing to move.

dequeue() and display() — same small patch

```
int dequeue(void)
{
    if (isEmpty())
    {
        printf("Queue Underflow\n");
        return -1;
    }

    int x = queue[front];

    if (front == rear)
    {
        front = -1;
        rear = -1;
    }
    else
        front = (front + 1) % MAX;

    return x;
}
```

- Everything is identical to Lecture 11 except one line: `front = (front + 1) % MAX`.
- The last-element reset (`front = rear = -1`) is UNCHANGED — empty still means empty the same way.
- `display` cannot use a simple for-loop anymore, since `front` can now be BIGGER than `rear` (the queue wraps). It needs a different walk — next slide.

display(): why the old loop breaks

The trap

front = 3, rear = 1 (wrapped).

for (i = front; i <= rear; i++)

checks $3 \leq 1$ — false immediately.

The loop body never runs. Silence, not a crash.

New idea: start at front and step with % MAX, and stop only when you PRINT the rear slot — not by comparing indices with <=.

```
void display(void)
{
    if (isEmpty()) { ... return; }

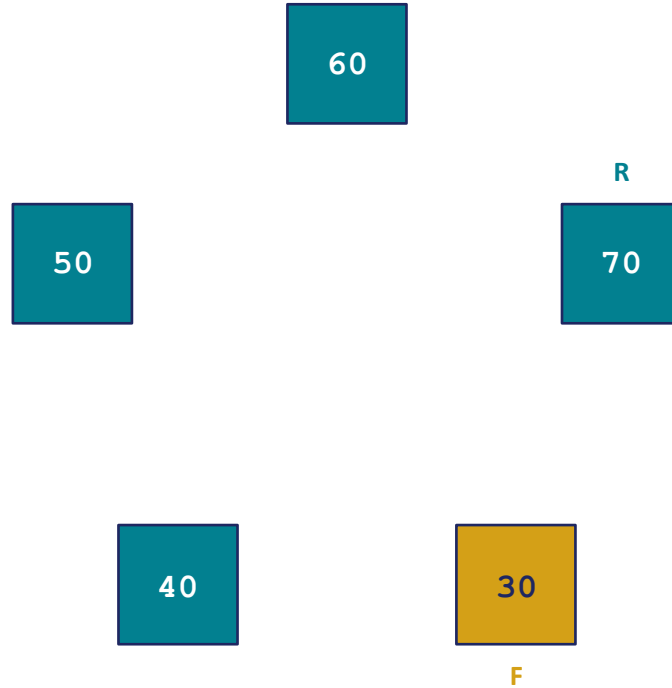
    int i = front;
    while (1)
    {
        printf("%d ", queue[i]);
        if (i == rear)
            break;
        i = (i + 1) % MAX;
    }
}
```

Trace: watch the wrap actually happen

Step	Array [0..4]	front	rear
fill 10, 20, 30, 40, 50	10 20 30 40 50	0	4
enqueue (60)	OVERFLOW — genuinely full	0	4
dequeue → 10	· 20 30 40 50	1	4
dequeue → 20	· · 30 40 50	2	4
enqueue (60)	60 · 30 40 50 (60 at index 0!)	2	0
enqueue (70)	60 70 30 40 50 (70 at index 1!)	2	1

The gold rows are the payoff: 60 lands at index 0, 70 at index 1 — slots Lecture 11 could never reach again.

The same trace, drawn as a clock



front (index 2) → rear (index 1), the long way around through 3, 4, 0

Where circular queue sits: quick comparison

Structure	Ends used	Array reuse	Common flaw
Stack	One (top)	N/A	None — one end never wastes space
Linear Queue	Two (front, rear)	Never (Lect. 11)	False overflow
Circular Queue	Two, wrapping	Full reuse (today)	isFull needs the % trick

Circular queues run continuously, all around you

CPU round-robin scheduling

Processes sit in a ring; the CPU serves each in turn and loops back to the first.

Audio / video buffering

A fixed-size ring buffer keeps writing new samples over old ones that were already played.

Traffic light control

A fixed cycle of signals repeats forever — conceptually a circular queue of states.

Keyboard / network buffers

A small ring buffer holds recent input; old, already-read data is safely overwritten.

Any system that runs forever, with fixed memory, tends to use a ring buffer — exactly this idea.

One operator fixes false overflow

- A circular queue treats the array as a ring: after index $\text{MAX} - 1$ comes index 0 again.
- The wrap rule: $\text{index} = (\text{index} + 1) \% \text{MAX}$ — used in both enqueue's rear step and dequeue's front step.
- `isFull` changes to $(\text{rear} + 1) \% \text{MAX} == \text{front}$ — full means rear is about to catch up to front.
- `display` must walk with the wrap rule too, stopping when it PRINTS rear, not by comparing indices.

EXIT TICKET $\text{MAX} = 4$. Starting empty: enqueue(1), enqueue(2), enqueue(3), dequeue, dequeue, enqueue(4), enqueue(5), enqueue(6). Give front and rear after each step, and say which enqueue call fails, if any.